```
data “amazon-ami” “ubuntu” {
      region = “us-east-1”
      filters = {
        virtualization-type = “hvm”
        name = “*ubuntu-xenial-*”
        root-device-type = “ebs”
      }
      owners = [“amazon”]
      most_recent = true
}

source “amazon-ebs” “glarimy-ubuntu” {
      source_ami = data.amazon-ami.ubuntu.id
      ...
}
...
```

The revised template has a new block named *data*. It defines a data source. We specified ‘amazon-ami’ as the data source type and ‘ubuntu’ as the label. This block uses several filters such as virtualization-type, name expression, and root device type. You may use any number of filters to narrow down the search. Packer will be connecting to the AWS AMI Registry to find all matching AMIs. The argument ‘most-recent=true’ selects the latest AMI among them and returns its ID and other information.

In the *source* block, we are referring to the selected ID dynamically with the following argument:

```
source_ami = data.amazon-ami.ubuntu.id
```

Run this template to see the results. Here are a few output lines of interest for us.

```
Prevalidating any provided VPC information
Prevalidating AMI Name: glarimy-ubuntu-1683014188
Found Image ID: ami-0b0ea68c435eb488d
```

You can observe from the above output that Packer found *ami-0b0ea68c435eb488d* as the source image matching our filters.

Customising the AMI

So far, we have just cloned the source AMI to build our own AMI. But this is not of good use. The real reason for us to use Packer is to build custom images. We have already talked about the immutable VMs. In this approach, we build a new AMI with all the required provisioning.

As you know, Ubuntu does not come with a Docker engine installed. Let us make our new images Docker-ready. For this, we need to add provisioning in the template file. Keep the *data* block and *source* blocks as they are, and edit the *build* block as follows in the *ami.pkr.hcl* file.

```
build {
      sources = [
        “source.amazon-ebs.glarimy-ubuntu”
      ]
      provisioner “shell” {
        inline = [“while [ ! -f /var/lib/cloud/instance/boot-
finished ]; do echo ‘Starting’; sleep 1; done”]
      }
      provisioner “shell” {
        scripts = [“docker.sh”]
      }
}
```

As you can observe, the *build* block has two *provisioner* blocks of type ‘shell’. It tells the Packer to run certain *shell* commands after deploying the source AMI on the instance and before creating the snapshot for the new AMI.

The *shell* commands can be supplied inline or in a separate file. For example, in the above template, the first provisioner is supplied with an inline command. This command essentially waits till the booting is complete so that we can run the other commands.

The second provisioner is supplied with a file named ‘docker.sh’. A file with that name must be present for the Packer to run. The content of the file has commands to install Docker.

```
sudo apt update
echo “Y” | sudo apt install docker.io
```

Running the above template gives an output from which certain lines are presented here.

```
Prevalidating any provided VPC information
Prevalidating AMI Name: glarimy-ubuntu-1683016633
Found Image ID: ami-0b0ea68c435eb488d
Launching a source AWS instance...
Instance ID: i-07c012aa3a2bcaee2
Waiting for instance (i-07c012aa3a2bcaee2) to become ready...
Using SSH communicator to connect: 3.90.36.190
Waiting for SSH to become available...
Connected to SSH!
Provisioning with shell script: /var/folders/ly/02f312xx3q5_
gqwlk82m1c7h0000gn/T/packer-shell1060700513
Starting
Starting
Starting
Starting
Provisioning with shell script: docker.sh
Unpacking docker.io (18.09.7-0ubuntu1~16.04.7)
Setting up docker.io (18.09.7-0ubuntu1~16.04.7)
```